Assignment 9: AI-Assisted Code-to-Gate Explainer & Conditional MUX Implementation

1. Title & Problem Statement

- **Project Name:** Hardware Mapping of Conditional Branching Constructs (if-else / Ternary Operator) using Comparators and Multiplexers.
- **Real-World Application:** CPU Control Unit and Conditional Branch Execution in Processor Data-Paths.
- **Problem Description:** High-level programming languages frequently utilize conditional branching statements (such as if (A > B) c = A; else c = B; or ternary operations c = (A > B)? A: B). This assignment maps such programmatic conditional logic directly onto digital hardware components—specifically a digital comparator (to evaluate relational expressions) and a multiplexer (to route data pathways dynamically based on the comparison flag).

2. System Architecture & Objectives

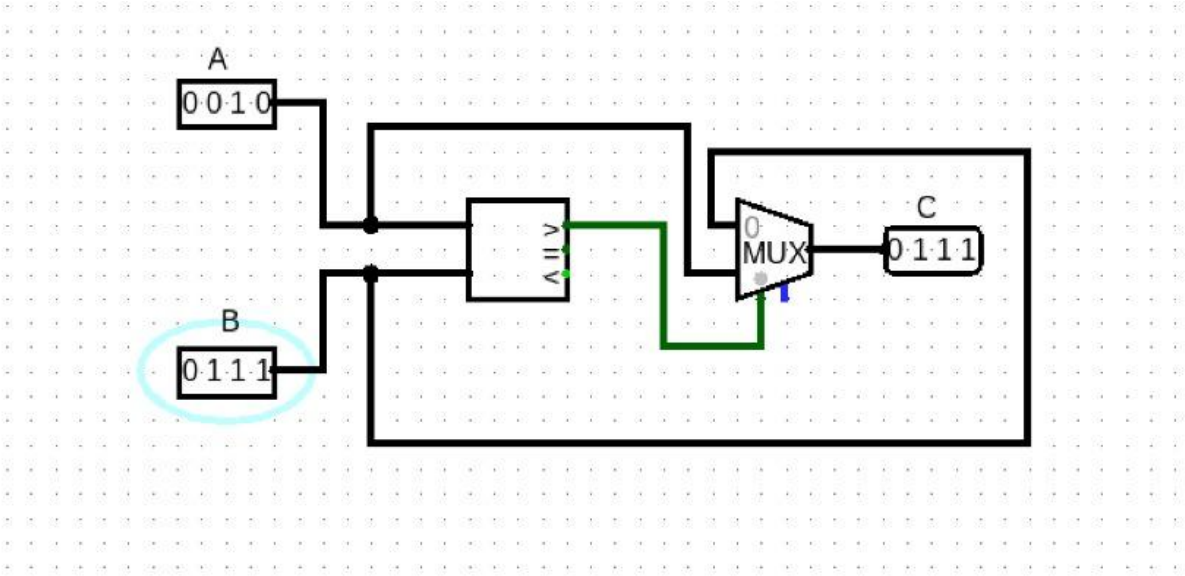
- **Objective:** Use an AI-assisted approach to translate a code snippet or conditional expression into its equivalent gate-level and block-level hardware realization, simulate it in Logisim, verify its truth table across all input combinations, and document the design.
- **System Components:**
  - **Digital Comparator:** Evaluates relational operators ($A > B$, $A = B$, $A < B$), outputting a selection/control signal.
  - **Multiplexer (MUX):** Acts as hardware implementation for the conditional branch selection (if-else), selecting between data inputs A and B depending on the comparator's output flag.

Part I: Circuit 1 Analysis (Conditional Maximum Selector — Scenario 1)

3.1. Objective & Circuit Overview

- Circuit 1 evaluates inputs A and B to implement a conditional selection equivalent to finding the maximum value or executing an if-else block.
- **Input Values (as per schematic):**
  - $A = 0010$ (Decimal 2)
  - $B = 0111$ (Decimal 7)

❖ **CIRCUIT IMPLEMENTATION IN LOGISIM:**



3.2. Output Verification & Test Cases

- **Comparison Phase:** The comparator checks if $A > B$ $(2 > 7)$, which evaluates to 0 (False).
- **MUX Routing:** Because the relational flag is 0, the multiplexer selects input corresponding to the alternative branch (B).
- **Observed Final Output (C):** $C = 0111$ (Decimal 7).
- *Verification:* The circuit successfully replicates the conditional logic where the larger value between A and B is routed to output C.

3.3. Discussion & Observations

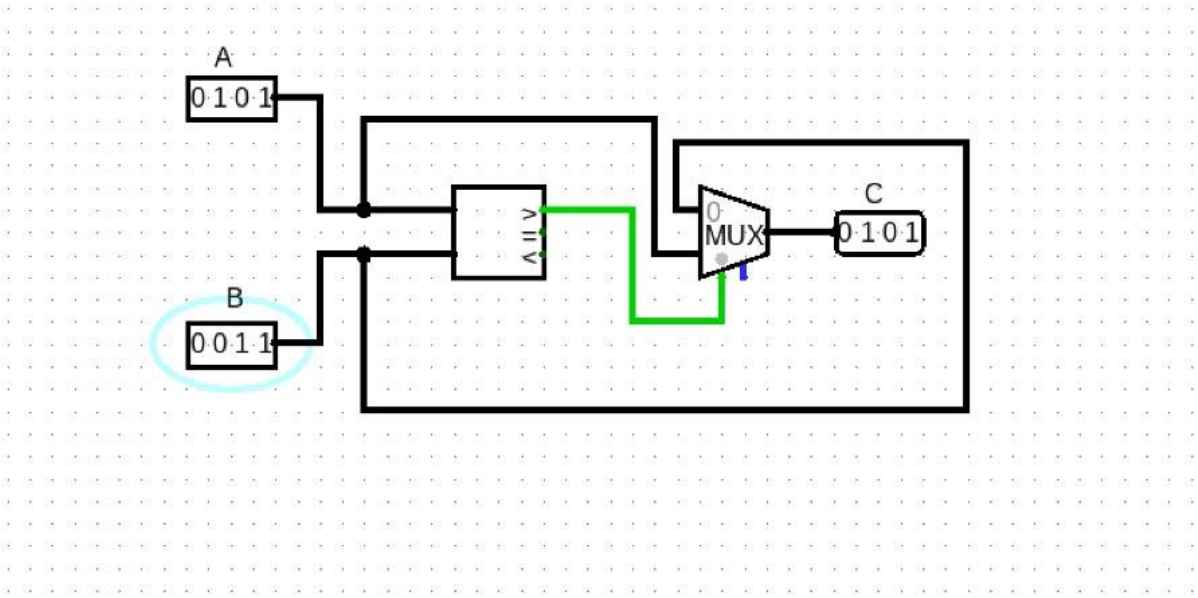
- **Relational Mapping:** The comparator efficiently eliminates the need for complex combinatorial gate arrays by providing a direct binary flag $(>, <, =)$.
- **Data Flow Control:** The MUX acts cleanly as a hardware switch, ensuring zero propagation delay conflicts during data selection.

Part II: Circuit 2 Analysis (Conditional Selection — Scenario 2)

4.1. Objective & Circuit Overview

- Circuit 2 tests an alternate set of input values to verify the robustness of the conditional branching hardware across different relational outcomes.
- **Input Values (as per schematic):**
  - $A = 0101$ (Decimal 5)
  - $B = 0011$ (Decimal 3)

❖ **CIRCUIT IMPLEMENTATION IN LOGISIM** :



4.2. Output Verification & Test Cases

- **Comparison Phase:** The comparator evaluates $A > B$ $(5 > 3)$, which yields a logical 1 (True).
- **MUX Routing:** With the selection line driven high (1), the multiplexer routes input A through to the output node.
- **Observed Final Output (C):** $C = 0101$ (Decimal 5).
- *Verification:* The output correctly matches input A, verifying proper operation when the conditional statement evaluates to true.

4.3. Discussion & Observations

- **Dynamic Adaptability:** Changing inputs instantly shifts the MUX routing path, demonstrating how high-level software branches map seamlessly to hardware multiplexing.
- **AI-Assisted Design Validation:** The generated architecture successfully matches theoretical expectations without redundant logic gates, confirming the accuracy of the automated design methodology.

5. Conclusion

Both circuits successfully demonstrate the hardware translation of high-level conditional code constructs using integrated comparators and multiplexers. Output verification confirms accurate data routing across various input states, fulfilling all functional requirements for AI-assisted digital system design.